EVALUACIÓN DE CALIDAD: PYRAMID TESTING

Estrategia de Pruebas Unitarias, Integración y End-to-End

Proyecto: MediControl
Entorno: Flutter + Node.js (SQLite)
Autor: Eduardo Rodríguez
Fecha: Octubre 2026

Fase 1. Diagnóstico de la Estrategia Actual

1. Funcionalidad Crítica Seleccionada: *Gestión, Agendamiento y Persistencia de Citas Médicas*. Esta funcionalidad comprende validaciones temporales (no permitir fechas pasadas), ordenamiento cronológico de la agenda, cálculo de la próxima consulta, serialización REST y persistencia transaccional en SQLite con modo WAL. Constituye el núcleo operativo para los pacientes en MediControl.

2. Diagnóstico Inicial de Pruebas en el Repositorio: Al auditar el proyecto inicial, se identificó un severo antipatrón de calidad conocido como **Cono de Helado (Ice Cream Cone)** o **Pirámide Invertida**: ausencia total de pruebas unitarias en lógica de negocio (0%) y pruebas de integración en backend (0%), contando únicamente con 2 pruebas básicas de UI en test/widget_test.dart.

Nivel de Prueba	Pruebas Iniciales	Tipo de Ejecución	Diagnóstico y Brecha Técnica
Unitarias (Base)	0 pruebas (0%)	Ninguna	Brecha crítica: La lógica de ordenamiento y validación no tenía ninguna cobertura aislada.
Integración (Conexiones)	0 pruebas (0%)	Ninguna	Brecha crítica: La interacción HTTP Express <-> SQLite no estaba automatizada.
E2E / UI (Cúspide)	2 pruebas (100%)	Manual / flutter test	Pruebas superficiales de widget en login sin validación de persistencia real.

■ Captura 1A: Pantalla de Inicio / Dashboard

Pegar captura de la pantalla DashboardScreen en ejecución móvil

[Captura de pantalla móvil]

Evidencia: Vista principal de MediControl con saludo al usuario, banner de próxima consulta y tarjetas de módulos.

■ Captura 1B: Agenda de Citas Médicas

Pegar captura de la pantalla AppointmentsScreen con citas cargadas

[Captura de pantalla móvil]

Evidencia: Pantalla AppointmentsScreen mostrando la lista de consultas agendadas y el botón flotante (+) Nueva cita.

Fase 2. Validación de la Base: Pruebas Unitarias

Se extrajo y formalizó el servicio desacoplado AppointmentLogic en lib/services/appointment_logic.dart. Se implementaron 3 pruebas unitarias exhaustivas en test/unit/appointment_logic_test.dart:

Caso de Prueba	Entradas y Condiciones	Resultado Esperado vs Obtenido	Estado
1. Caso Válido	Fecha futura (28 oct 2026), todos los campos completos.	isValid: true, errorMessage: null. Formateo legible '28 oct 2026'.	**PASÓ**
2. Caso Límite	Fecha en el instante actual exacto; 2 citas separadas por 1 minuto; descarte de citas pasadas en findNextAppointment.	Ordenamiento cronológico exacto respetando minutos; selección estricta de la próxima cita no vencida.	**PASÓ**

3. Caso Inválido / Error	Campos con solo espacios en blanco; fecha en el pasado (2 días antes).	isValid: false con mensajes: 'Completa todos los campos' y 'La fecha y hora no pueden ser en el pasado'.	**PASÓ**

```
$ flutter test test/unit/appointment_logic_test.dart 00:00 +0: Caso Válido: Valida exitosamente una cita con datos completos y fecha futura 00:00 +1: Caso Límite: Cita en el límite temporal actual y ordenamiento por minutos 00:00 +2: Caso Inválido / Error: Rechaza campos en blanco y fechas en el pasado 00:00 +3: All tests passed!
```

■ Captura 2: Ejecución de Pruebas Unitarias en Terminal / IDE

Pegar captura de la terminal o de VS Code / Android Studio ejecutando las pruebas unitarias

[Espacio reservado para evidencia visual del proyecto • Ajustar imagen a este marco]

Evidencia: Consola de comandos ejecutando 'flutter test test/unit/appointment_logic_test.dart' con 3 pruebas aprobadas al 100%.

Fase 3. Validación de Conexiones: Pruebas de Integración

Se construyó una suite de pruebas de integración en backend/test/appointments_integration.test.js utilizando el ejecutor nativo node:test y aserciones estrictas de node:assert. Las pruebas validan la interacción **real** entre el servidor HTTP Express y la base de datos física SQLite en disco (sin simular la base de datos):

Prueba de Integración	Componentes Reales Evaluados	Datos y Comprobación Real	Estado
Integración 1: Flujo Correcto	Cliente HTTP Fetch <-> Express Router <-> SQLite WAL (better-sqlite3)	POST /api/appointments inserta 'Dra. Valery'. Se verifica con SELECT directo en SQLite que la fila existe físicamente con su ID generado y coincide en GET.	**PASÓ**
Integración 2: Manejo de Errores e Integridad	Validadores de Express <-> Transacciones SQLite	POST sin specialty devuelve HTTP 400 y SELECT COUNT(*) confirma 0 alteraciones en BD. DELETE de ID inexistente devuelve 404 Not Found. DELETE de ID creado purga la fila.	**PASÓ**

```
$ npm test (backend) ■ Fase 3: Pruebas de Integración - API REST & SQLite Database ✔ Prueba Integración 1 (Flujo Correcto): Inserta vía POST HTTP y comprueba persistencia física en SQLite (80.3ms) ✔ Prueba Integración 2 (Manejo de Errores): Rechaza payload incompleto (400) y maneja 404 en borrado (29.2ms) ✔ Fase 3: Pruebas de Integración (117ms) - tests 2, pass 2, fail 0
```

■ Captura 3: Ejecución de Pruebas de Integración y Persistencia en SQLite

Pegar captura de la terminal de Node.js ejecutando las pruebas de integración o visor de base de datos SQLite

[Espacio reservado para evidencia visual del proyecto • Ajustar imagen a este marco]

Evidencia: Salida de consola de 'npm test' evidenciando las llamadas HTTP REST y la persistencia transaccional en el archivo medicontrol.sqlite.

Fase 4. Validación de un Flujo Completo: End-to-End (E2E)

Se implementó una prueba E2E 100% automatizada en test/e2e/e2e_appointment_flow_test.dart mediante flutter_test. El test recorre el ciclo de vida completo de un usuario en la plataforma sin intervención manual:

- **1. Autenticación:** Inicia en LoginScreen, llena credenciales válidas y pulsa 'Iniciar sesión'.
- **2. Acceso y Navegación:** Valida bienvenida en Dashboard y hace tap en la pestaña 'Citas' del NavigationBar.
- **3. Formulario Modal:** Abre el modal 'Nueva cita' mediante el FloatingActionButton e ingresa 'Dr. E2E Automatizado' y detalles.

- **4. Persistencia en UI:** Envía el formulario y comprueba la aparición inmediata de la tarjeta en la agenda ordenada.
- **5. Operación de Baja:** Localiza el botón de papelera del registro específico y comprueba su eliminación definitiva de pantalla.

■ Captura 4A: Formulario Modal de Nueva Cita

Pegar captura del modal showModalBottomSheet con los campos del formulario llenos

[Captura de pantalla móvil]

Evidencia: Modal interactivo desplegado desde el FAB (+) con los campos Médico, Especialidad, Ubicación y Motivo diligenciados.

■ Captura 4B: Cita Agendada y Reflejada en la Lista

Pegar captura de la pantalla mostrando la nueva tarjeta agregada a la lista de citas

[Captura de pantalla móvil]

Evidencia: Tarjeta de la nueva cita 'Dr. E2E Automatizado' agregada a la agenda y lista para ser eliminada.

```
$ flutter test test/e2e/e2e_appointment_flow_test.dart 00:00 +0: loading test/e2e/e2e_appointment_flow_test.dart 00:00 +0: Fase 4: Prueba E2E - Flujo Crítico Completo (Login -> Navegación -> Alta de Cita -> Verificación -> Baja) 00:02 +1: All tests passed!
```

■ Captura 4C: Evidencia de Ejecución de la Prueba E2E Automatizada

Pegar captura de la terminal ejecutando el test E2E de Flutter

[Espacio reservado para evidencia visual del proyecto • Ajustar imagen a este marco]

Evidencia: Consola de pruebas ejecutando el test 'e2e_appointment_flow_test.dart' con el recorrido de login, navegación, guardado y borrado exitoso.

Fase 5. Auditoría y Conclusión Técnica

1. Consolidación de Resultados y Reestructuración de la Pirámide:
Tras la implementación de las tres fases, se logró corregir el antipatrón inicial y conformar una **Pirámide de Pruebas Saludable**:

Nivel de Prueba	Antes (Antipatrón)	Ahora (Pirámide de Calidad)	Tiempo de Ejecución	Eficacia y Retorno
Unitarias (Base)	0 (0%)	**3 pruebas (37.5%)**	< 1 segundo	Máxima velocidad, aísla fallos de lógica al instante.
Integración (Cuerpo)	0 (0%)	**2 pruebas (25.0%)**	~1.1 segundos	Garantiza consistencia real HTTP y atomicidad SQLite.
E2E / UI (Cúspide)	2 (100%)	**3 pruebas (37.5%)**	~2.2 segundos	Garantiza la experiencia del usuario y flujos transversales.

2. Capa Más Débil Identificada Inicialmente: La base unitaria y la capa de integración estaban al 0%. Cualquier error de cálculo o corrupción en SQLite solo era detectable mediante interacción visual manual en el emulador.

3. Dos Mejoras Técnicas Priorizadas por Riesgo:
• **Mejora 1 (Riesgo Alto - Automatización CI/CD):** Implementar un pipeline en GitHub Actions que ejecute flutter test test/unit/ y npm test en cada Pull Request para evitar regresiones antes del merge.
• **Mejora 2 (Riesgo Medio - Aislamiento de Base de Datos de Pruebas):** Configurar en el backend una variable SQLITE_PATH=:memory: para las pruebas de integración, evitando modificar el archivo SQLite de desarrollo.

4. Matriz de Ejecución según Etapa de Desarrollo:
• **En cada cambio de código (Pre-commit / Pull Request):** Ejecutar las pruebas unitarias y de integración rápida (< 2 seg). Proporcionan retroalimentación instantánea al desarrollador.
• **Antes de una entrega o pase a producción (Pre-release):** Ejecutar la suite E2E completa, verificación de flujos críticos de usuario y auditoría de regresión visual.